AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

# DOKUMENTACJA PROJEKTOWA

Programowanie zaawansowane

## Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Jakub Pyrdoł

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

# Spis treści

**1. Ogólne określenie wymagań** **3**
1.1. Zakres funkcjonalny . . . 3
1.2. Przykład . . . 3
1.3. Instalacja . . . 4

**2. Analiza problemu** **5**
2.1. Zastosowanie i reprezentacja danych . . . 5
2.2. Dodawanie i usuwanie elementów . . . 5
2.3. Ręczny przykład działania algorytmu . . . 6
2.4. Factory, Iterator i wspólny kursor . . . 7
2.5. Złożoność obliczeniowa . . . 7

**3. Projektowanie** **8**
3.1. Narzędzia i konfiguracja budowania . . . 8
3.2. Struktura plików i odpowiedzialności klas . . . 9
3.3. Diagram klas . . . 10
3.4. Wykorzystanie Git i GitHub . . . 10

**4. Implementacja** **12**
4.1. Inicjalizacja listy i tworzenie węzłów . . . 12
4.2. Dodawanie i usuwanie elementów . . . 13
4.3. Iterator i wyświetlanie listy . . . 15
4.4. Nawigacja wspólnym kursorem . . . 15
4.5. Czyszczenie i zwalnianie pamięci . . . 16
4.6. Wyniki działania programu . . . 17
4.7. Ograniczenia bieżącej implementacji . . . 18

**5. Wnioski** **19**

**Literatura** **20**

**Spis rysunków** **21**

**Spis tabel** **22**

**Spis listingów** **23**

# 1. Ogólne określenie wymagań

Celem projektu jest opracowanie listy dwukierunkowej przechowującej liczby całkowite, z wykorzystaniem języka C++14. Każdy węzeł zawiera wartość oraz wskaźniki do obu sąsiadów. Klasa listy zarządza elementami, prosta fabryka odpowiada za ich tworzenie, a iterator umożliwia przechodzenie po kolekcji. Do rejestrowania zmian wykorzystano Git i repozytorium GitHub.

Dokumentacja opisuje kod przekazany w archiwum projektu [1], uzupełniony opisem Doxygen [2]. Obejmuje wymagania, analizę algorytmu, organizację klas, konfigurację budowania, implementację, rzeczywiste wyniki uruchomienia i wnioski.

## 1.1. Zakres funkcjonalny

Program udostępnia:

- dodawanie liczb na początku, na końcu i pod wskazanym indeksem;
- usuwanie elementów z początku, z końca i z wybranej pozycji;
- wyświetlanie całej listy w obu kierunkach;
- przechodzenie wspólnym kursorem do następnego lub poprzedniego węzła;
- czyszczenie listy oraz uzyskiwanie iteratorów przez `begin()` i `end()`.

Wartości i indeksy mają typ `int`. Indeksy liczone są od zera. Lista jest tworzona dynamicznie i nie ma zadeklarowanego stałego limitu liczby elementów; możliwość utworzenia węzła zależy od dostępnej pamięci. Program nie wymaga zewnętrznych bibliotek C++.

Plik `main.cpp` zawiera ustaloną sekwencję wywołań demonstracyjnych. Aplikacja działa w konsoli, wypisuje wyniki na standardowe wyjście i nie pobiera danych od użytkownika. Zmiana demonstracji wymaga zmiany wartości lub wywołań w funkcji `main()` i ponownego zbudowania programu.

## 1.2. Przykład

Wywołania `push_back(10)`, `push_front(5)` i `push_wherever(1, 7)` tworzą kolejno listy $[10]$, $[5, 10]$ i $[5, 7, 10]$. Przejście od początku daje wartości 5, 7, 10, a od końca 10, 7, 5. Szczegółowy przykład przepinania wskaźników przedstawiono w rozdziale 2.

## 1.3. Instalacja

Do zbudowania kodu potrzebny jest kompilator obsługujący C++14. Projekt zawiera plik `CMakeLists.txt`, który wymaga CMake w wersji co najmniej 4.1. Narzędzia i ustawienia środowiska autora zestawiono w rozdziale 3. Do samego uruchomienia skompilowanego programu nie są potrzebne pliki danych ani dodatkowa konfiguracja.

W CLion należy otworzyć katalog zawierający `CMakeLists.txt`, wybrać skonfigurowany zestaw narzędzi C++ i poczekać na odczytanie projektu. Następnie buduje się i uruchamia cel `ListaDwukierunkowa_Pyrdol_Jakub`. Odtwarzając projekt w innym katalogu, należy wygenerować nowy katalog budowania zamiast używać załączonego katalogu `cmake-build-debug`, który zawiera ścieżki z komputera autora.

Przykładowe polecenia CMake, wykonywane w katalogu projektu C++:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --config Debug
```

W konfiguracji Ninja plik wykonywalny powstaje w katalogu `build`. W systemie Windows ma rozszerzenie `.exe`; generator wielokonfiguracyjny może umieścić go w podkatalogu `Debug`.

Kod można także skompilować bez CMake, wywołując GCC bezpośrednio:

```
g++ -std=c++14 main.cpp DoublyLinkedList.cpp Node.cpp Factory.cpp Iterator.cpp
    -o lista
```

W systemie Linux program uruchamia się poleceniem `./lista`, a w systemie Windows, po nadaniu plikowi nazwy `lista.exe`, poleceniem `.\lista.exe`. Polecenia dotyczą źródeł aplikacji; ich kopia wykorzystywana w dokumentacji znajduje się w katalogu `kod` tej dokumentacji.

# 2. Analiza problemu

## 2.1. Zastosowanie i reprezentacja danych

Lista dwukierunkowa jest przydatna w uporządkowanych kolekcjach, w których potrzebne są przejścia do poprzedniego i następnego elementu. Może stanowić podstawę kolejek lub mechanizmów nawigacji po sekwencji. W tym projekcie służy do przedstawienia operacji na liczbach całkowitych i współpracy klas odpowiedzialnych za dane, alokację oraz iterowanie.

Węzeł `Node` zawiera pola `value`, `next` i `prev`. Klasa `DoublyLinkedList` przechowuje wskaźniki `head`, `tail`, licznik `size`, obiekt `factory` oraz wskaźnik `cursor`. Dla poprawnej struktury połączeń zachodzą następujące zależności:

- pusta lista ma `head == nullptr`, `tail == nullptr` i `size == 0`;
- pierwszy element ma `prev == nullptr`, a ostatni `next == nullptr`;
- dla sąsiadów $A$ i $B$, połączeniu `A->next == B` odpowiada połączenie `B->prev == A`;
- pole `size` odpowiada liczbie węzłów osiągalnych od początku.

Kursor służy do nawigacji krokowej i nie zmienia kolejności elementów. Usunięcie wskazywanego węzła zeruje kursor, a czyszczenie listy resetuje go przed usuwaniem elementów.

## 2.2. Dodawanie i usuwanie elementów

Dodawanie na początku łączy nowy węzeł z dotychczasowym pierwszym elementem i zmienia `head`. Na końcu analogicznie zmienia się `tail`. Dla pustej listy nowy węzeł staje się oboma końcami. Każde dodanie zwiększa `size` o jeden.

Wstawianie wewnętrzne wymaga przejścia od `head` do podanej pozycji i dołączenia nowego węzła do obu sąsiadów. Usuwanie łączy sąsiadów, zwalnia węzeł przez `delete` i zmniejsza licznik. Usuwanie na końcach aktualizuje wskaźniki skrajne; po usunięciu ostatniego elementu oba są puste.

**Tab. 2.1.** Zachowanie metod przy indeksach granicznych

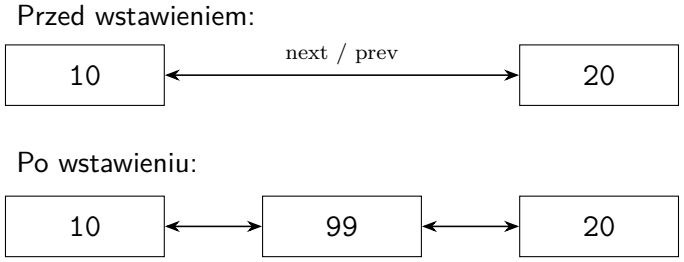
| **Warunek** | **Działanie implementacji** |
|---|---|
| `push_wherever`, `index < 0` | Powrót bez zmiany listy. |
| `push_wherever`, `index == 0` | Wywołanie `push_front`. |
| `push_wherever`, `index >= size` | Dopisanie na końcu przez `push_back`. |
| `pop_wherever`, indeks poza $[0, size - 1]$ | Powrót bez zmiany listy. |
| Usuwanie z pustej listy | Powrót bez usunięcia elementu i bez komunikatu błędu. |

## 2.3. Ręczny przykład działania algorytmu

Rozważmy listę $[5, 10, 20, 30]$ i wywołanie `push_wherever(2, 99)`. Po dwóch przejściach przez `next` zmienna `current` wskazuje węzeł o wartości 20. Jego poprzednikiem jest węzeł o wartości 10. Po utworzeniu węzła o wartości 99 program ustawia:

1. `newNode->next = current;`
2. `newNode->prev = current->prev;`
3. `current->prev->next = newNode;`
4. `current->prev = newNode.`

Licznik rośnie z 4 do 5, a lista przyjmuje postać $[5, 10, 99, 20, 30]$. Rysunek 2.1 pokazuje lokalną zmianę połączeń.




**Rys. 2.1.** Wstawienie wartości 99 pomiędzy węzły o wartościach 10 i 20

W sekwencji demonstracyjnej po usunięciu początku pozostaje $[10, 99, 20, 30]$, po usunięciu końca $[10, 99, 20]$, a wywołanie `pop_wherever(1)` daje $[10, 20]$. Usuwany

węzeł o wartości 99 zostaje pominięty przez połączenia obu sąsiadów, a jego pamięć jest zwalniana.

## 2.4. Factory, Iterator i wspólny kursor

Klasa `Factory` tworzy węzły przez `new Node(value)`, oddzielając alokację od przepinania połączeń. Konstruktor węzła zapisuje wartość i ustawia oba wskaźniki sąsiadów na `nullptr`.

Iterator przechowuje pozycję w polu `current`. Operatory `++` i `--` korzystają z połączeń sąsiadów, `*` odczytuje wartość, a `!=` porównuje pozycje. Początek wskazuje `head`, pozycja końcowa ma wartość `nullptr`, a przejście wstecz zaczyna się od `tail`.

Wspólny `cursor` przy wartości `nullptr` rozpoczyna przejście od początku albo końca, zależnie od kierunku. Kolejne wywołania przemieszczają tę samą pozycję. Wyjście poza listę wypisuje komunikat i pozostawia pusty kursor; następne wywołanie zaczyna przejście ponownie.

## 2.5. Złożoność obliczeniowa

Niech $n$ oznacza liczbę elementów, a $i$ poprawny indeks pozycji wewnętrznej. Oszacowania w tabeli 2.2 wynikają z liczby przejść i zmian wskaźników w kodzie; nie są pomiarami czasu wykonania. Pomijają koszt przydziału pamięci i fizycznego wypisywania znaków.

**Tab. 2.2.** Złożoność operacji na liście

| **Operacja** | **Czas** | **Uzasadnienie** |
|---|---|---|
| Dodawanie i usuwanie na końcach | $O(1)$ | Dostęp przez `head` lub `tail`. |
| Wstawienie lub usunięcie wewnętrznego elementu | $O(i)$, do $O(n)$ | Przejście zawsze od początku. |
| Wyświetlenie całej listy i `clear_list()` | $O(n)$ | Odwiedzenie wszystkich węzłów. |
| Krok kursora, operatory iteratora, `begin()`, `end()` | $O(1)$ | Operacje na pojedynczej pozycji. |

Lista zajmuje $O(n)$ pamięci, a operacje $O(1)$ pamięci pomocniczej.

# 3. Projektowanie

## 3.1. Narzędzia i konfiguracja budowania

Kod podzielono na pliki nagłówkowe i źródłowe. Konfiguracja CMake ustawia `CMAKE_CXX_STANDARD` na 14, dodaje bieżący katalog do ścieżek nagłówków i buduje jeden program z pięciu plików `.cpp`. Zachowane pliki konfiguracji budowania wskazują środowisko autora przedstawione w tabeli 3.1.

**Tab. 3.1.** Środowisko odczytane z przekazanego projektu

| **Element** | **Ustawienie** |
|---|---|
| System i architektura | Windows, AMD64. |
| Środowisko programistyczne | CLion 2025.3.1.1. |
| Kompilator | GNU GCC 13.1.0, platforma MinGW. |
| Standard języka | C++14; w zapisanym budowaniu flaga `-std=gnu++14`. |
| System budowania | CMake 4.1.2, generator Ninja. |
| Konfiguracja | Debug, flaga `-g`. |
| Biblioteki | Biblioteka standardowa C++; brak dodatkowych zależności projektu. |
| Dokumentacja kodu | Doxygen 1.18.0. |

Listing 1 przedstawia pełną konfigurację projektu. Wymaganie CMake 4.1 wynika z jego pierwszej instrukcji. Projekt nie definiuje osobnych celów testowych CTest ani opcji optymalizacji dla listy.

Komentarze Doxygen wykorzystują znaczniki `@file`, `@brief`, `@param` i `@return` do opisu plików, klas, argumentów i wyników metod. Dostarczony dokument PDF zawiera indeks klas, indeks plików i opisy publicznego interfejsu. Szczegółowe zachowanie indeksów, kursora i zarządzania pamięcią w niniejszym opracowaniu ustalono również bezpośrednio z definicji metod.

```
cmake_minimum_required(VERSION 4.1)
project(ListaDwukierunkowa_Pyrdol_Jakub)

set(CMAKE_CXX_STANDARD 14)

include_directories(.)

add_executable(ListaDwukierunkowa_Pyrdol_Jakub main.cpp DoublyLinkedList.cpp
    Node.cpp Factory.cpp Iterator.cpp)
```

**Listing 1.** Konfiguracja CMake projektu

## 3.2. Struktura plików i odpowiedzialności klas

**Tab. 3.2.** Podział kodu aplikacji

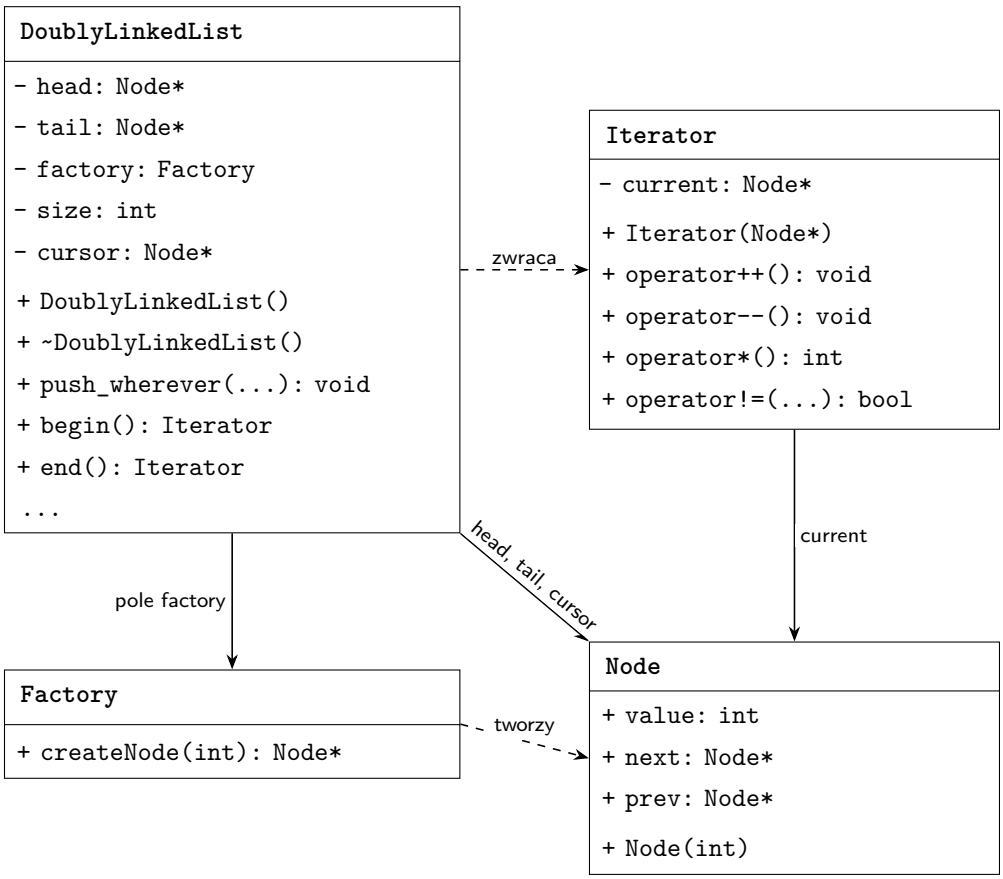
| **Pliki** | **Odpowiedzialność** |
|---|---|
| `Node.h`, `Node.cpp` | Wartość elementu, połączenia i inicjalizacja węzła. |
| `Factory.h`, `Factory.cpp` | Tworzenie nowych węzłów. |
| `Iterator.h`, `Iterator.cpp` | Bieżąca pozycja oraz operatory przejścia, odczytu i porównania. |
| `DoublyLinkedList.h`, `DoublyLinkedList.cpp` | Zarządzanie listą, jej rozmiarem i kursorem. |
| `main.cpp` | Ustalony scenariusz demonstracyjny i wypisywanie wyników. |
| `CMakeLists.txt` | Wybór standardu i plików budowanego programu. |

Pola `head`, `tail`, `size`, `factory` i `cursor` w klasie listy są prywatne. W iteratorze prywatna jest pozycja `current`. Klasa `Node` udostępnia swoje pola publicznie. Klasy współpracują przez obiekty i wskaźniki; kod nie zawiera hierarchii dziedziczenia.

Lista jest właścicielem swoich węzłów i zwalnia je w destruktorze. Konstruktor kopiujący oraz kopiujący operator przypisania są oznaczone przez `= delete`. Zapobiega to tworzeniu obiektów współdzielących te same węzły przez niezamierzone kopiowanie wskaźników.

## 3.3. Diagram klas

Rysunek 3.1 przedstawia pola i wybrane metody czterech klas. Uwzględnia konstruktor, destruktor listy i wspólny kursor nawigacji. Strzałki opisują zależności wynikające z kodu, a znak wielokropka oznacza skrócenie listy metod.




**Rys. 3.1.** Podział klas i zależności w zrealizowanym projekcie

Metody wyświetlania całej listy tworzą lokalne iteratory. Nie korzystają ze wspólnego pola `cursor`, dlatego pełne wyświetlenie nie zmienia pozycji używanej w nawigacji krokowej. Fabryka tworzy węzły, ale nie zarządza ich późniejszym zwalnianiem; usuwanie wykonuje klasa listy.

## 3.4. Wykorzystanie Git i GitHub

Git rejestruje kolejne wersje projektu, a GitHub przechowuje jego zdalne repozytorium. W konfiguracji przekazanego repozytorium adres `origin` wskazuje https://github.com/pyr-jak/ListaDwukierunkowa_Pyrdol_Jakub.git. Lokalna historia zawiera gałęzie `main` i `docs` oraz ich scalenie. Zestawienie wybranych zapisów przedstawia tabela 3.3; dane pochodzą z historii dołączonej do archiwum [3].

**Tab. 3.3.** Wybrane etapy zapisane w historii Git

| **Data** | **Commit** | **Etap** |
|---|---|---|
| 18.10.2026 | `b5cedb0` | Inicjalizacja projektu CLion. |
| 18.10.2026 | `5a00c2d` | Klasy Factory, Iterator i Node. |
| 20.10.2026 | `6c36a17` | Dodanie osobnej gałęzi. |
| 25.10.2026 | `6380f4d` | Konstruktor listy i iterator. |
| 25.10.2026 | `a54cded` | Operacje dodawania. |
| 25.10.2026 | `b903712` | Operacje usuwania i demonstracja. |
| 25.10.2026 | `5882246` | Wyświetlanie wstecz i czyszczenie. |
| 25.10.2026 | `7fd975d` | Nawigacja next/prev i demonstracja. |
| 25.10.2026 | `d9140cc` | Komentarze do dokumentacji Doxygen. |
| 25.10.2026 | `677004b` | Poprawki kursora i zarządzania pamięcią. |

Stan plików można sprawdzić przez `git status`, różnice przez `git diff`, a historię przez `git log --oneline --graph --all`. Przygotowanie wersji obejmuje `git add` i `git commit`; `git push` przekazuje zapisane zmiany do repozytorium zdalnego. Historia umożliwia prześledzenie rozwoju od klas pomocniczych do pełnej sekwencji operacji na liście.

# 4. Implementacja

Listingi w tym rozdziale są odczytywane bezpośrednio z kopii plików projektu w katalogu `kod`. Numery wierszy odpowiadają plikom dołączonym do tej dokumentacji. Pozwala to powiązać opis z definicjami metod.

## 4.1. Inicjalizacja listy i tworzenie węzłów

Konstruktor listy zeruje jej rozmiar i ustawia `head`, `tail` oraz `cursor` na `nullptr`. Utworzenie samej listy nie alokuje jeszcze węzłów.

```
14 DoublyLinkedList::DoublyLinkedList()
15 {
16     head = nullptr;
17     tail = nullptr;
18     cursor = nullptr;
19     size = 0;
20 }
```

**Listing 2.** Konstruktor klasy DoublyLinkedList

Konstruktor węzła zapisuje przekazaną liczbę i inicjalizuje połączenia. Fabryka deleguje do niego utworzenie obiektu na stercie.

```
17 Node::Node(int value)
18 {
19     this->value = value;
20     this->next = nullptr;
21     this->prev = nullptr;
22 }
```

**Listing 3.** Inicjalizacja węzła

```
11 Node* Factory::createNode(int value)
12 {
13     return new Node(value);
14 }
```

**Listing 4.** Tworzenie obiektu przez klasę Factory

## 4.2. Dodawanie i usuwanie elementów

Metody `push_front` i `push_back` rozróżniają listę pustą i niepustą. W pierwszym przypadku nowy węzeł staje się jednocześnie początkiem i końcem. W drugim dołączany jest do właściwego sąsiada, a odpowiedni wskaźnik skrajny zostaje zmieniony.

Metoda `push_wherever` najpierw rozpatruje indeks ujemny, początek oraz pozycję równą lub większą od rozmiaru. Pozostały przypadek obsługuje pętla przechodząca do wskazanego węzła i cztery przypisania połączeń opisane w rozdziale 2.

```
101 void DoublyLinkedList::push_wherever(int index, int value)
102 {
103     if (index < 0)
104     {
105         return;
106     }
107 
108     if (index == 0)
109     {
110         push_front(value);
111         return;
112     }
113 
114     if (index >= size)
115     {
116         push_back(value);
117         return;
118     }
119 
120     Node* newNode = factory.createNode(value);
121     Node* current = head;
122 
123     for (int i = 0; i < index; i++)
124     {
125         current = current->next;
126     }
127 
128     newNode->next = current;
129     newNode->prev = current->prev;
130     current->prev->next = newNode;
131     current->prev = newNode;
132     size++;
133 }
```

**Listing 5.** Wstawianie elementu pod wskazanym indeksem

Usuwanie na początku i końcu sprawdza, czy lista jest pusta. Po zmianie wskaźnika skrajnego program poprawia połączenie sąsiada albo zeruje drugi koniec listy. Następnie zwalnia pamięć usuniętego węzła. Jeżeli usuwany węzeł był wskazywany przez `cursor`, przed `delete` kursor zostaje ustawiony na `nullptr`. Każde skuteczne usunięcie zmniejsza `size` o jeden.

Metoda `pop_wherever` odrzuca indeks spoza zakresu i przekazuje przypadki skrajne do `pop_front` lub `pop_back`. Dla pozycji wewnętrznej przepisuje połączenia sąsiadów, usuwa węzeł i zmniejsza licznik.

```
200 void DoublyLinkedList::pop_wherever(int index)
201 {
202     if (index < 0 || index >= size)
203     {
204         return;
205     }
206 
207     if (index == 0)
208     {
209         pop_front();
210         return;
211     }
212 
213     if (index == size - 1)
214     {
215         pop_back();
216         return;
217     }
218 
219     Node* current = head;
220 
221     for (int i = 0; i < index; i++)
222     {
223         current = current->next;
224     }
225 
226     current->prev->next = current->next;
227     current->next->prev = current->prev;
228     if (cursor == current)
229     {
230         cursor = nullptr;
231     }
232 
233     delete current;
234     size--;
235 }
```

**Listing 6.** Usuwanie elementu o podanym indeksie

## 4.3. Iterator i wyświetlanie listy

Metoda `begin()` tworzy iterator wskazujący `head`. Metoda `end()` zwraca iterator z `nullptr`. Iterator wykonuje operacje bezpośrednio na swoim wskaźniku. Przykład operatora przejścia przedstawia listing 7.

```
19 void Iterator::operator++()
20 {
21     current = current->next;
22 }
```

**Listing 7.** Przejście iteratora do następnego węzła

Operator `--` przypisuje `current->prev`, operator `*` zwraca `current->value`, a `!=` porównuje dwa pola `current`. Operatory przejścia zwracają `void`, a wyłuskanie liczbę, więc interfejs jest własnym, uproszczonym iteratorem projektu.

Przed wyświetleniem lista jest sprawdzana pod kątem pustej struktury. Wtedy pojawia się komunikat `Lista jest pusta`. Dla niepustej listy pętla używa iteratora i wypisuje kolejne wartości oddzielone spacjami.

```
240 void DoublyLinkedList::display_list()
241 {
242     if (head == nullptr)
243     {
244         std::cout << "Lista jest pusta\n";
245         return;
246     }
247
248     for (Iterator it = begin(); it != end(); ++it)
249     {
250         std::cout << *it << " ";
251     }
252     std::cout << std::endl;
253 }
```

**Listing 8.** Wyświetlanie listy od początku

Wyświetlenie odwrotne działa podobnie, lecz rozpoczyna się od `Iterator(tail)` i używa `--it`. Warunek zakończenia nadal porównuje iterator z `end()`, czyli pozycją `nullptr`.

## 4.4. Nawigacja wspólnym kursorem

Metoda `display_next_element` sprawdza najpierw pustą listę. Dla niepustej struktury rozpoczyna przejście od `head`, jeżeli kursor nie wskazuje węzła, albo przechodzi przez `cursor->next`. Po wyjściu za koniec wypisuje `End of the list`; w przeciwnym razie podaje bieżącą wartość.

```
276 void DoublyLinkedList::display_next_element()
277 {
278     if (head == nullptr)
279     {
280         std::cout << "List is empty" << std::endl;
281         return;
282     }
283 
284     if (cursor == nullptr)
285     {
286         cursor = head;
287     }
288     else
289     {
290         cursor = cursor->next;
291     }
292 
293     if (cursor == nullptr)
294     {
295         std::cout << "End of the list" << std::endl;
296         return;
297     }
298 
299     std::cout << "Next element: " << cursor->value << std::endl;
300 }
```

**Listing 9.** Przejście wspólnego kursora do następnego elementu

Metoda `display_prev_element` korzysta z `tail` i `prev`. Komunikaty programu brzmią dokładnie `Previous element:` oraz `Beggining of the list`; zachowano ich oryginalną pisownię. Obie metody używają tego samego pola, dlatego zmiana kierunku oznacza przejście od ostatnio wskazywanego węzła. Dla pustej listy wypisują `List is empty`.

Po usunięciu wskazywanego węzła kursor jest pusty. Dla niepustej listy następne wywołanie nawigacji rozpocznie przejście od `head` lub `tail`, zależnie od kierunku. Usunięcie innego węzła zachowuje bieżącą pozycję kursora.

## 4.5. Czyszczenie i zwalnianie pamięci

Metoda `clear_list` najpierw zeruje `cursor`, a następnie usuwa pierwszy element tak długo, aż `head` przyjmie wartość `nullptr`. Wykorzystuje już istniejącą obsługę usuwania, zamiast powtarzać przepinanie połączeń. Na końcu wypisuje `Clearing Completed`, również dla początkowo pustej listy.

```
334 void DoublyLinkedList::clear_list()
335 {
336     cursor = nullptr;
337 
338     while (head != nullptr)
339     {
340         pop_front();
341     }
342     std::cout<<"Clearing Completed"<<std::endl;
343 }
```

**Listing 10.** Czyszczenie wszystkich węzłów listy

Usuwane węzły są zwalniane przez `delete` w metodach usuwających. Jawne wywołanie `clear_list()` pozwala opróżnić obiekt i używać go ponownie. Niezależnie od tego destruktor usuwa węzły pozostałe przy końcu życia listy. Funkcja `main()` nadal wykonuje jawne czyszczenie na końcu demonstracji; późniejsze wywołanie destruktora zastaje już pustą listę.

Destruktor usuwa pozostałe węzły przez kolejne wywołania `pop_front()`. Sprzątanie odbywa się automatycznie przy końcu życia obiektu i nie wypisuje dodatkowego komunikatu. Zablokowanie kopiowania chroni przed współdzieleniem tych samych węzłów przez różne listy.

```
24 DoublyLinkedList::~DoublyLinkedList()
25 {
26     while (head != nullptr)
27     {
28         pop_front();
29     }
30 }
```

**Listing 11.** Automatyczne zwalnianie węzłów w destruktorze

## 4.6. Wyniki działania programu

Funkcja `main()` przedstawia podstawowe operacje listy: dodawanie i usuwanie elementów, wyświetlanie w obu kierunkach, nawigację kursorem oraz czyszczenie struktury.

Listing 12 przedstawia rzeczywisty standardowy strumień wyjściowy programu. Widać kolejność elementów po dodawaniu i usuwaniu, przejście odwrotne, komunikaty graniczne kursora i zakończenie czyszczenia.

```
Testing push_back 10 20 30
Testing push_front 5 10 20 30
Testing push_wherever 5 10 99 20 30
 REVERSED LIST: 30 20 99 10 5
Testing pop_front 10 99 20 30
Testing pop_back 10 99 20
Testing pop_wherever 10 20
Next element: 10
Next element: 20
End of the list
Previous element:20
Previous element:10
Beggining of the list
Clearing Completed
```

**Listing 12.** Wynik uruchomienia oryginalnego programu main.cpp

**Tab. 4.1.** Kolejne stany listy w programie demonstracyjnym

| **Etap** | **Stan listy** |
|---|---|
| Trzy wywołania `push_back` | $[10, 20, 30]$ |
| `push_front(5)` | $[5, 10, 20, 30]$ |
| `push_wherever(2, 99)` | $[5, 10, 99, 20, 30]$ |
| `pop_front()` | $[10, 99, 20, 30]$ |
| `pop_back()` | $[10, 99, 20]$ |
| `pop_wherever(1)` | $[10, 20]$ |
| `clear_list()` | [ ] |

## 4.7. Ograniczenia bieżącej implementacji

**Zakres iteratora.** Operatory `++`, `--` i `*` nie sprawdzają `current == nullptr`. Wolno ich używać tylko dla istniejącego węzła. W szczególności `--end()` nie rozpoczyna przejścia od końca. Iterator zwraca wartość, a nie referencję do elementu, i nie udostępnia pełnego zestawu cech iteratorów biblioteki standardowej.

**Obsługa błędów.** Błędne indeksy są ignorowane albo, przy wstawianiu zbyt daleko, oznaczają dopisanie na końcu. Metody nie zwracają statusu tych decyzji. Fabryka nie przechwytuje wyjątku braku pamięci zgłaszanego przez `new`. Są to właściwości aktualnego interfejsu, które trzeba uwzględnić przy jego wykorzystaniu w większym programie.

# 5. Wnioski

W projekcie zrealizowano listę dwukierunkową liczb całkowitych z dodawaniem, usuwaniem, przejściami w obu kierunkach i czyszczeniem. Podział na klasy `Node`, `Factory`, `Iterator` i `DoublyLinkedList` oddziela przechowywanie danych, tworzenie węzłów, odczyt pozycji oraz zarządzanie połączeniami. Konstruktor listy ustala jednoznaczny stan początkowy, a licznik rozmiaru umożliwia rozpatrywanie operacji indeksowych.

Zastosowanie wskaźników początku i końca pozwala wykonywać operacje skrajne w czasie stałym. Operacje wewnętrzne wymagają przejścia do pozycji, dlatego ich czas rośnie liniowo w najgorszym przypadku. Iterator upraszcza kod wyświetlania i umożliwia wykorzystanie tej samej zasady porównania pozycji końcowej przy przejściu w obu kierunkach.

Uruchomienie oryginalnej demonstracji dało oczekiwane kolejności elementów po kolejnych zmianach listy. Kursor jest zerowany po usunięciu wskazywanego elementu i po czyszczeniu listy. Destruktor zwalnia pozostałe węzły, a zablokowanie kopiowania zapobiega przypadkowemu współdzieleniu ich przez różne obiekty.

Dalszym rozszerzeniem może być dobór bliższego końca przy dojściu do indeksu oraz pełniejszy interfejs iteratora.

Historia Git dokumentuje stopniowe dodawanie konstruktorów, operacji dodawania, usuwania i nawigacji. Repozytorium GitHub umożliwia zachowanie kolejnych wersji, a Doxygen porządkuje opisy klas i metod. Połączenie tych narzędzi z opisem algorytmu i zapisanymi wynikami uruchomienia tworzy dokumentację bieżącej wersji projektu wraz z jej rozpoznanymi ograniczeniami.

# Literatura

[1] Jakub Pyrdoł. ListaDwukierunkowa_Pyrdol_Jakub. kod źródłowy projektu, 2026. Pliki C++, konfiguracja CMake i zachowane ustawienia budowania; stan materiałów: 25 października 2026 r.

[2] Jakub Pyrdoł. Algorytm Listy Dwukierunkowej. dokumentacja kodu. Dokumentacja wygenerowana przez Doxygen 1.18.0, 2026. Dostarczony plik doxygen.pdf; opisy czterech klas oraz plików projektu.

[3] Jakub Pyrdoł. Repozytorium projektu listy dwukierunkowej. https://github.com/pyr-jak/ListaDwukierunkowa_Pyrdol_Jakub, 2026.

# Spis rysunków

2.1. Wstawienie wartości 99 pomiędzy węzły o wartościach 10 i 20 . . . . . 6
3.1. Podział klas i zależności w zrealizowanym projekcie . . . . . 10

# Spis tabel

2.1. Zachowanie metod przy indeksach granicznych . . . . . 6
2.2. Złożoność operacji na liście . . . . . 7
3.1. Środowisko odczytane z przekazanego projektu . . . . . 8
3.2. Podział kodu aplikacji . . . . . 9
3.3. Wybrane etapy zapisane w historii Git . . . . . 11
4.1. Kolejne stany listy w programie demonstracyjnym . . . . . 18

# Spis listingów

1. Konfiguracja CMake projektu . . . . . 9
2. Konstruktor klasy DoublyLinkedList . . . . . 12
3. Inicjalizacja węzła . . . . . 12
4. Tworzenie obiektu przez klasę Factory . . . . . 12
5. Wstawianie elementu pod wskazanym indeksem . . . . . 13
6. Usuwanie elementu o podanym indeksie . . . . . 14
7. Przejście iteratora do następnego węzła . . . . . 15
8. Wyświetlanie listy od początku . . . . . 15
9. Przejście wspólnego kursora do następnego elementu . . . . . 16
10. Czyszczenie wszystkich węzłów listy . . . . . 17
11. Automatyczne zwalnianie węzłów w destruktorze . . . . . 17
12. Wynik uruchomienia oryginalnego programu main.cpp . . . . . 18